

Go Programming - Basics

Concepts of Programming Languages

Sebastian Macke

Rosenheim Technical University

Last Lecture:

- Introduction
- The many ways to characterize a language
- Go Introduction
- Characteristics of a Successful Language

-- Miro Board

2

Characteristics of a Successful Language (Suggestion)

- Simplicity and readability (Small instruction set and easy syntax)
- Abstraction (types/classes, Generics, Procedures)
- Efficiency
- Reliability (compatibility, error checking, no memory leaks)
- Dependency management
- Good support and documentation (public compilers, tutorials, community, IDE)
- Orthogonality

What is an Orthogonal language

- **Def 1:** A language is orthogonal if its features are built upon a small, mutually **independent** set of primitive operations.
- **Def 2:** We can combine the majority of the language constructs in many ways without any side effects
- **Def 3:** A language is non-orthogonal when combinations of features behave inconsistently or unexpectedly.
- Other definitions:

stackoverflow.com/questions/3272019/is-java-orthogonal (<https://stackoverflow.com/questions/3272019/is-java-orthogonal>)

- Associated with simplicity and consistency.
- The more orthogonal the design, the fewer exceptions.
- The Go language was designed with orthogonality in mind.

Orthogonal Design by an example

- Type System is a "feature"
- Functions are a "feature"
- Types are used in functions as parameters or return values.
- Orthogonal would mean, that the type system is independent of the functions.
- E. g. The behaviour of the type is not influenced by a function or vice-versa.

5

Types VS. Functions (Non-Orthogonal Design by example)

- E. g. in C, the type system is not orthogonal to the function system.

```
// Ints are allowed to return
int returnint() {
    return 1;
}

// Structs are allowed to return
typedef struct{} structdef;
structdef returnstruct() {
    structdef mystruct;
    return mystruct;
}

// Arrays are not allowed to return
int[3] returnarray() {
    static int myarray[3] = {1, 2, 3};
    return myarray;
}
```

- But arrays in structs are allowed to return

Every number in Java is the same. Right?

Numbers and classes might be non-orthogonal in Java.

```
Integer a = 1000;  
Integer b = 1000;  
System.out.println(a == b); // false  
  
Integer a = 1;  
Integer b = 1;  
System.out.println(a == b); // true
```

7

Other non-orthogonal examples

- In C, "a++" behaves different for an integer a than for a pointer a.

-- Go doesn't allow this operation.

- In Java, there are three kinds of loops: for, while, do-while.

-- In Go, there is only one loop: for

- In Java the way variables are stored depends on their types.

```
int x; // x is a value  
MyClass y; // y is a reference
```

-- Go has pointers and values. But the way they are stored is the same.

8

Golang



#cyberpanda @c0dehard

Golang

- Go is an open source programming language for distributed and parallel systems
- Go addresses the problems of C ++ backend development at Google
- The Go core team is prominent: Robert Griesheimer (Hotspot VM), Ken Thompson (UX / B) and Rob Pike (UTF-8)
- Go is actively developed since 2008
- Go is **THE** language behind the Cloud Native Stack

www.cncf.io/ (<https://www.cncf.io/>)



- All essential components are written in Go: Docker, Kubernetes, Etcd, Prometheus, Grafana -> An important reason to take a closer look at Go

TypeScript will be written in Go

- Some build systems for the JavaScript ecosystem are already build in Go: esBuild, vite
- Anders Hejsberg: Microsoft Technical Fellow and lead architect of TypeScript. Original designer of C#, Delphi, and Turbo Pascal.

devblogs.microsoft.com/typescript/typescript-native-port/ (<https://devblogs.microsoft.com/typescript/typescript-native-port/>)

11

Tools

go env Print Go environment information

go run file.go just runs the code. Can be multiple files, too.

go build downloads dependencies and builds the executable, but needs a go.mod file.

go tool dist list list all supported platforms

env GOOS=darwin GOARCH=arm64 go build Build for Apple ARM CPU computers.

go fmt formats your files

go mod init creates a go.mod module file

- module file contains language version information, module name and dependencies

go mod tidy updates the go mod file if you add more dependencies

12

Golang compiler source code compiles in 36 seconds

Go compiles fast

```
> time ./make.bash
Building Go cmd/dist using /opt/homebrew/Cellar/go/1.23.0/libexec. (go1.23.0 darwin/arm64)
Building Go toolchain1 using /opt/homebrew/Cellar/go/1.23.0/libexec.
Building Go bootstrap cmd/go (go_bootstrap) using Go toolchain1.
Building Go toolchain2 using go_bootstrap and Go toolchain1.
Building Go toolchain3 using go_bootstrap and Go toolchain2.
Building packages and commands for darwin/arm64.
---
Installed Go for .../go-src/go
Installed commands in .../go-src/go/bin
*** You need to add .../go-src/go/bin to your PATH.
./make.bash 167,89s user 43,26s system 580% cpu 36,367 total
```

13

Packages

```
package main

import (
    "fmt"
)

func main() {
    fmt.Printf("Hello %s", "Programming with Go \xE2\x98\xAF\n") // \xE2\x98\xAF -> ☹
    fmt.Printf("Hello %s", "Programming with Go ☹\n")
}
```

- The entry point for Go executable is always the function main in package main (main.main)
- New packages need a new directory and can then be referenced
- **Public functions begin with a capital letter**

Primitive Types

- Basic building block of a programming language
- They are not defined in terms of other data types
- Usually designed with the CPU architecture in mind

Primitive Types

Golang	Java	Comment
bool	boolean	
string	String	
int		machine-sized in Go
int8	byte	
int16	short	
int32	int	
int64	long	
uint	-	unsigned int
uint8	-	
uint16	-	
uint32	-	
uint64	-	
uintptr	-	
byte	-	alias for uint8
rune	char	alias for int32, represents a Unicode code point. In Java usually 2 bytes.
float32	float	
float64	double	
complex64 // complex		
complex128		
struct {x,z,y int}	class	

- In Go the type of a variable is behind the name!

Complex Numbers

```
package main

import (
    "fmt"
    "math/cmplx"
)

func main() {
    c := 3 + 4i
    fmt.Printf("c=%v\n", c)
    r, θ := cmplx.Polar(c) // Go supports variables with unicode characters
    fmt.Printf("r=%v, θ=%v\n", r, θ)
}
```

- The assignment ':= ' is used for short variable declarations. The type is deduced implicitly. Also called "type inference"
- Go itself (like Python) is not designed for number crunching

All primitive types are value types

Value types are primitive data types (int, double, boolean, etc.) that directly store their values in memory rather than referencing an object.

```
func foo(x int) {  
    x = 2  
}  
  
func bar(s string) {  
    s = "world"  
}  
  
func main() {  
    x := 1  
    foo(x)  
    fmt.Println(x) // 1  
  
    s := "Hello"  
    bar(s)  
    fmt.Println(s) // Hello  
}
```

The many ways of variable declarations in Go

```
var foo int // default value zero
foo = 32

var foo int = 32

var foo = 32

foo := 32 // infers (guesses) the type definition of type int

foo := int(32)

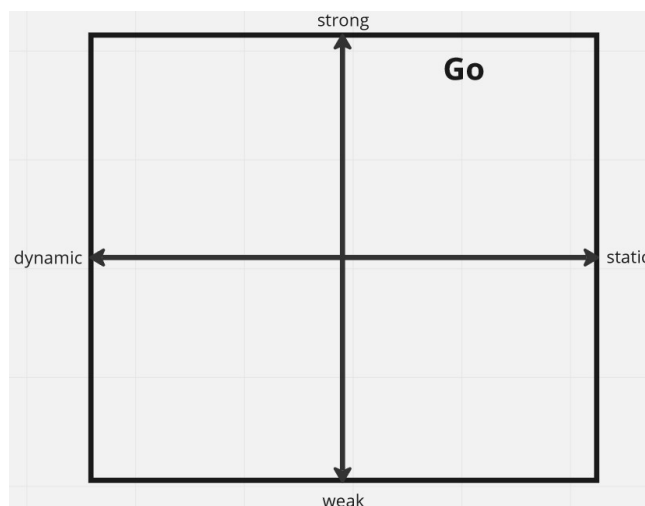
foo, ok := 32, true // foo is an int 32, ok is a boolean with value true
```

Weak typing vs. Strong typing I

Languages differ of how serious they handle the type

- type conversions implicit or explicit
- strict or loose typing rules at compile time
- dynamic types (runtime types) allowed or not

Weak typing vs. Strong typing II



- weakly typed: allow implicit conversions
- strongly typed: don't allow implicit conversions
- statically typed: type checking at compile time
- dynamically typed: type checking at runtime

Go uses a strong typing system. But why?

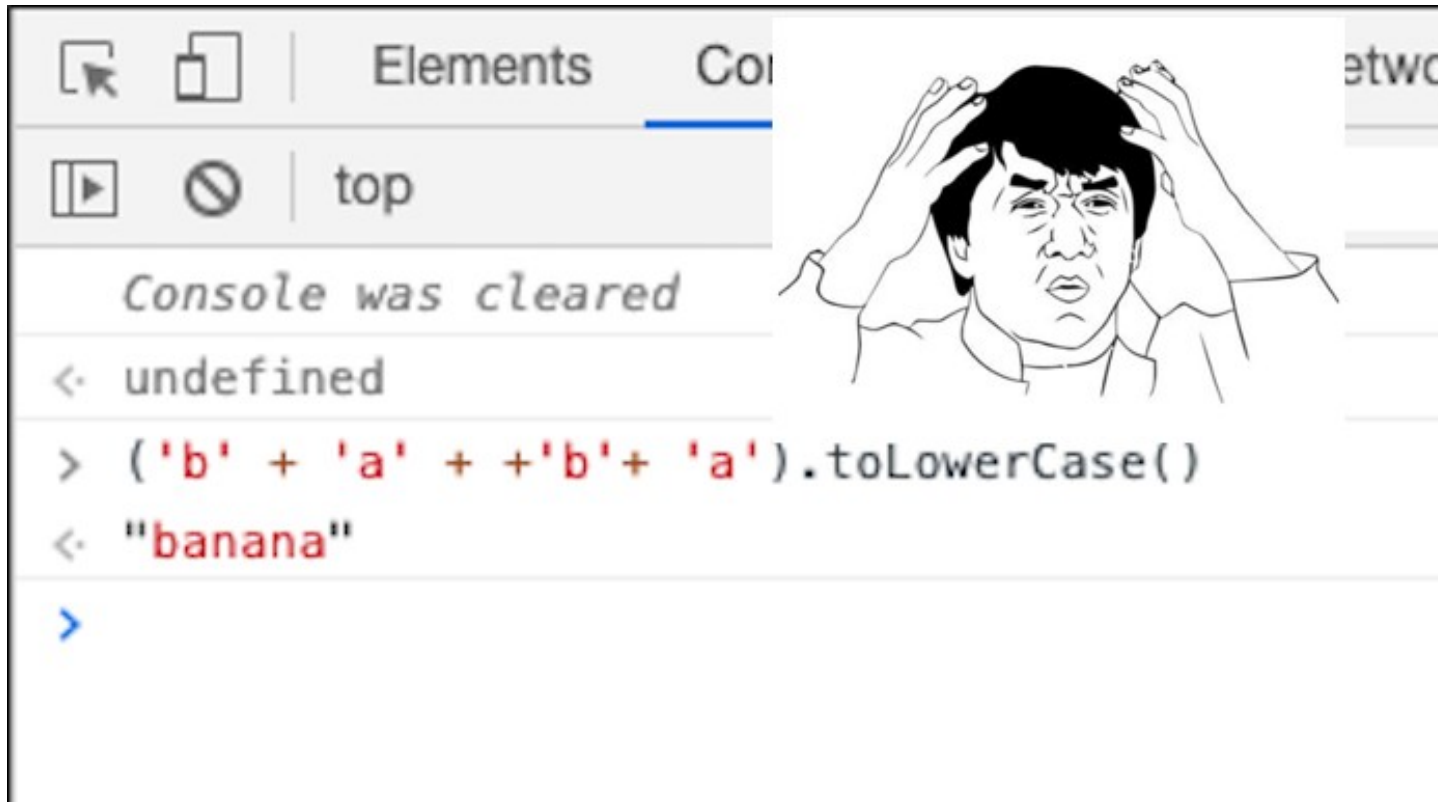
Where implicit type conversion in C goes wrong

```
int main() {  
    signed int a = -1;  
    unsigned int b = 0;  
  
    if (a < b) {  
        printf("a is smaller than b\n");  
    }  
  
    return 0;  
}
```

Also: Change int to char

22

Javascript is very creative with type coercion



Strong typing in Go

```
package main

import "fmt"

func main() {
    var a int32 = -1
    var b uint32 = 2

    if a < int32(b) {
        fmt.Printf("a is smaller than b")
    }
}
```

- In Go there is no implicit type cast.
- The compiler will complain if you try to compare different types.
- You have to do the type conversion explicitly.

Dynamically typing in Python

```
x = 10          # x is an int
x = "hello"     # now x is a str
```

```
def greet(**kwargs):
    for key, value in kwargs.items():
        print(f"{key} = {value}")

greet(name="Alice", age=30, city="Munich")
```

```
def dynamic(value):
    if value > 0:
        return "positive"
    else:
        return -1
```

Go has also a dynamic type: any

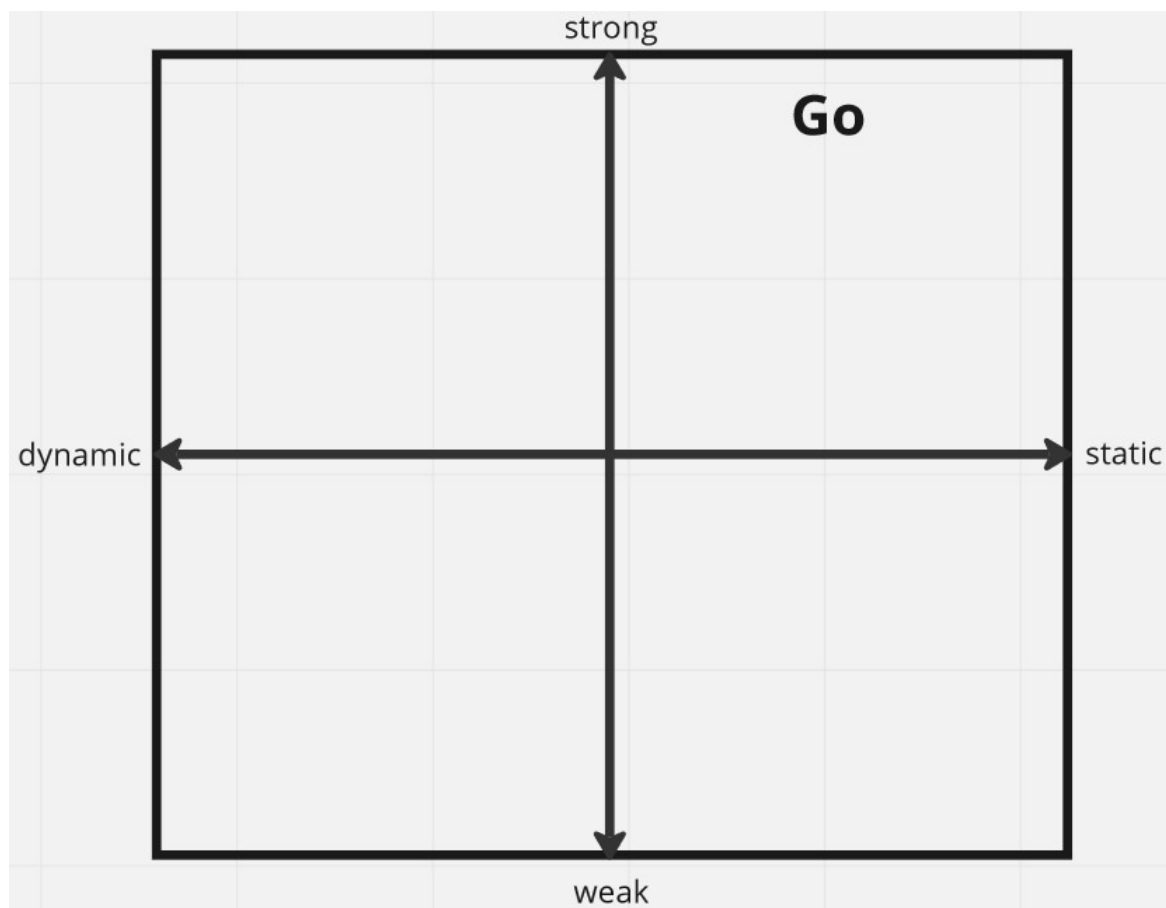
```
func PrintVariableDetails(v any) {
    typeof := reflect.TypeOf(v)
    fmt.Printf("The variable with type '%s' has the value '%v'\n", typeof.Name(), v)
}

func main() {
    var someValue any
    someValue = 2
    PrintVariableDetails(someValue)

    someValue = "abcd"
    PrintVariableDetails(someValue)

    if tmp, ok := someValue.(string); ok {
        fmt.Println("someValue is a string and has the value", tmp)
    }
}
```

Type systems in other languages



- Miro board

Golang - complex types

Maps

- Maps are Go's built-in associative data type (sometimes called hashes or dicts in other languages).

```
package main

import "fmt"

func main() {
    m := make(map[string]string) // Initialize an empty map
    m["foo"] = "bar"             // insert a key-value pair into map

    value, ok := m["asd"] // check if key is present, return parameters can be ignored with "_"
    fmt.Println(ok, value) // returns false, value is just an empty string ""

    value, ok = m["foo"] // returns true, value contains "bar"
    fmt.Println(ok, value)

    for k, v := range m {
        fmt.Println(k, v)
    }
}
```

Map is a reference type

```
package main

import "fmt"

func main() {
    m1 := map[string]int{"x": 1, "y": 2}
    m2 := m1 // m2 points to the same map as m1
    m2["x"] = 100

    fmt.Println("m1:", m1) // map[x:100 y:2]
    fmt.Println("m2:", m2) // map[x:100 y:2]
}
```

30

Arrays and Slices

- Arrays have a fixed length and can not be resized
- Slices are dynamic arrays and can be resized (`append( )`)
- Slices are views to underlying arrays or can be created dynamically
- The underlying array grows automatically (if needed)
- Both have a length and a capacity: What does that mean?

31

Arrays and Slices

```
package main

import "fmt"

func main() {
    arr := [5]int{1, 2, 3, 4, 5} // create fixed array
    fmt.Println(arr)

    s := arr[0:2] // create a slice out of the array. Otherwise via "make([2]int)" for a new array
    fmt.Println(s)

    fmt.Println(len(s), cap(s)) // ==> 2, 5
    s[0] = 100                  // a slice is just a reference
    fmt.Println(arr)

    s = append(s, 8)
    s = append(s, 9)
    s = append(s, 10)
    s = append(s, 11)

    fmt.Println(len(s), cap(s)) // ==> 6, 10
    fmt.Println(arr)
}
```

Non-Orthogonal: Arrays are value types, but slices are reference types. But they use the same syntax.

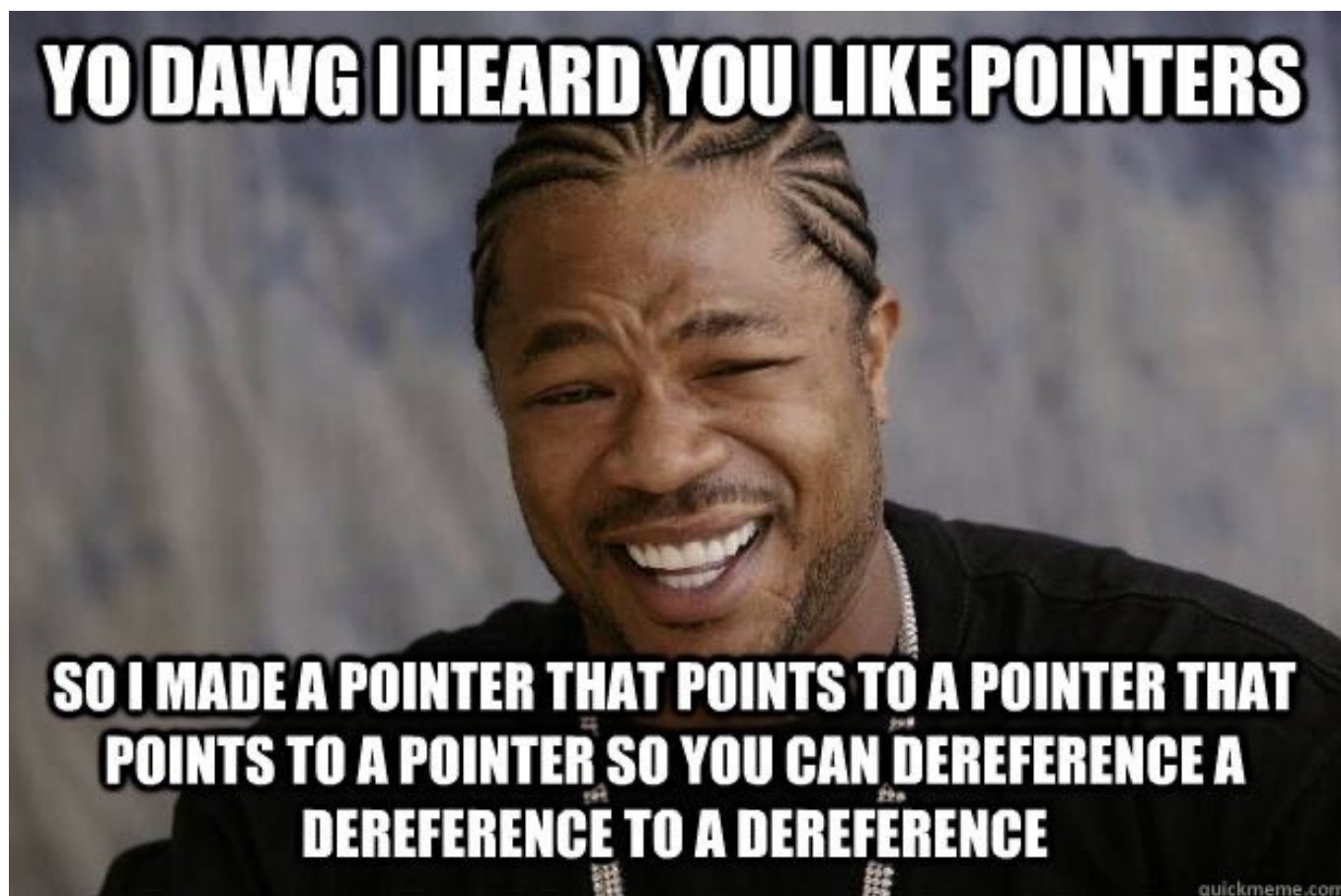
```
// function taking an array
func modifyArray(a [3]int) {
    a[0] = 99 // modifies only the local copy
}

// function taking a slice
func modifySlice(s []int) {
    s[0] = 99 // modifies the underlying array
}

func main() {
    // Arrays
    arr := [3]int{1, 2, 3}
    modifyArray(arr)
    fmt.Println("Array after modifyArray:", arr) // [1 2 3] unchanged

    // Slices
    slice := []int{1, 2, 3}
    modifySlice(slice)
    fmt.Println("Slice after modifySlice:", slice) // [99 2 3] changed
}
```

Pointers



Pointer I

A pointer is a variable which contains a memory address

```
func main() {  
    num := 42    // implicit type "int" with value 42  
    ptr := &num // implicit type of "pointer to type int points to num"  
    //var num int = 42 // explicit type definition  
    //var ptr *int = &num // explicit type definition  
  
    fmt.Println("Value of num:", num)  
    // Expected Output: Value of num: 42  
  
    fmt.Println("Address of num:", &num)  
    // Expected Output: Address of num: 0xSOME_MEMORY_ADDRESS (this will vary every run)  
  
    fmt.Println("Address stored in ptr:", ptr)  
    // Expected Output: Address stored in ptr: 0xSOME_MEMORY_ADDRESS (same as address of num)  
  
    fmt.Println("Value via pointer:", *ptr)  
    // Expected Output: Value via pointer: 42  
  
    *ptr = 43  
    fmt.Println("Value of num:", num)  
    // Expected output: New value in num = 43  
}
```

Pointer I

```
func swap0(x, y int) (int, int) {  
    return y, x  
}
```

```
func swap1(x, y int) {  
    x, y = y, x  
}
```

```
func swap2(x *int, y *int) {  
    *x, *y = *y, *x  
}
```

```
func swap3(x **int, y **int) {  
    *x, *y = *y, *x  
}
```

- The double assignment saves a variable (tmp)!
- Pointers are transferred as well as values by copy

Pointer II

```
func main() {  
    var a, b = 1, 2  
    fmt.Printf("Initial : a=%d, b=%d\n", a, b)  
  
    a, b = b, a  
    fmt.Printf("After a,b = b,a : a=%d, b=%d\n", a, b)  
  
    swap0(a, b)  
    fmt.Printf("After swap0(a,b) : a=%d, b=%d\n", a, b)  
  
    a, b = swap0(a, b)  
    fmt.Printf("After a,b = swap0(a,b) : a=%d, b=%d\n", a, b)  
  
    swap1(a, b)  
    fmt.Printf("After swap1(a,b) : a=%d, b=%d\n", a, b)  
  
    swap2(&a, &b)  
    fmt.Printf("After swap2(&a,&b) : a=%d, b=%d\n", a, b)  
  
    pa, pb := &a, &b  
    swap3(&pa, &pb)  
    fmt.Printf("After swap3(&pa, &pb): a=%d, b=%d, *pa=%v, *pb=%v\n", a, b, *pa, *pb)  
}
```

Don't do pointer arithmetic in Go

```
func main() {  
    z := [2]int64{10, 20} // fixed size array  
    y := &z[0]             // pointer to first entry in array  
    fmt.Println(y)  
    y = (*int64)(unsafe.Add(unsafe.Pointer(y), 8)) // 64 Bit has 8 bytes  
    fmt.Println(y)  
    *y = 30 // changes second parameter in the array z  
    fmt.Println(z)  
}
```

- Because of garbage collection anything can happen here! Even more unsafe than in C!

Pointer arithmetic is a "DON'T DO" in Go!

- There is some unsafe pointer arithmetic as in C / C ++

Functions and Control Structures: Example Palindrome

```
// IsPalindrome implementation. Does only work for 1-Byte UTF-8 chars (ASCII).  
func IsPalindrome(word string) bool {  
    for pos := 0; pos < len(word)/2; pos++ {  
        if word[pos] != word[len(word)-pos-1] {  
            return false  
        }  
    }  
    return true  
}
```

- The return type of a function is behind the parameter list
- Conditions (if, for) are not clipped with ()
- if, for ... statements need curly braces {}

Go directly supports Unit Tests via "go test"

```
// palindrome_test.go
func TestPalindrome(t *testing.T) {
    if !IsPalindrome("") {
        t.Error("isPalindrome('') should be true. But is false.")
    }
    if !IsPalindrome("o") {
        t.Error("isPalindrome('o' should be true. But is false.")
    }
    if !IsPalindrome("oto") {
        t.Error("isPalindrome('oto' should be true. But is false.")
    }
    if IsPalindrome("ottos") {
        t.Error("isPalindrome('ottos' should be false. But is true.")
    }
}
```

- The unit test for a file is located in a `_test.go` file of the same name
- `TestXY` functions are called automatically
- The test context `testing.T` controls the execution
- Has anyone done Test Driven Development?

Functions and Control Structures: Example Palindrome (UTF-8)

```
// IsPalindrome2 is using runes. This works for all UTF-8 chars (SBC, MBC).
func IsPalindrome2(word string) bool {
    var runes = []rune(word)
    for pos, ch := range runes {
        if ch != runes[len(runes)-pos-1] {
            return false
        }
    }
    return true
}
```

- The rune type is an alias for int32 and can store all UTF-8 characters
- `[]rune(string)` converts a string into a slice of runes
- The range operator has two return values: the position and the current value

```
pos, ch := range runes
```

Functions and Control Structures: Example Palindrome (Reverse)

```
// IsPalindrome3 is implemented by reusing Reverse(). Reverse works for UTF-8 chars.  
func IsPalindrome3(word string) bool {  
    return strings.Reverse(word) == word  
}
```

- Strings, arrays are compared in Go with ==
- Slices, Maps are compared with cmp.Equal

Exercise 2.1

[github.com/s-macke/concepts-of-programming-languages/blob/master/docs/exercises/
Exercise2.1.md](https://github.com/s-macke/concepts-of-programming-languages/blob/master/docs/exercises/Exercise2.1.md) (<https://github.com/s-macke/concepts-of-programming-languages/blob/master/docs/exercises/Exercise2.1.md>)

43

Maps and Slices - Example Book Index

Types can be defined with the keyword "type"

```
// Page contains an array of words.
type Page []string

// Book is an array of pages.
type Book []Page

// Index contains a list of pages for each word in a book.
type Index map[string][]int

// MakeIndex generates an index structure
func MakeIndex(book Book) Index {
    ....
}
```

The Flag API simplifies Command Line Utilities

```
import (  
    "flag"  
    "fmt"  
)  
  
// Simple test for the Go flag API.  
func main() {  
    // construct a string flag with a default ip address and a description.  
    ip := flag.String("ip", "192.168.1.1", "Overrides the default IP address.")  
    port := flag.String("port", "8080", "Overrides the default listen port.")  
    flag.Parse()  
  
    fmt.Println("\nDefault value for IP: " + *ip)  
    fmt.Println("\nDefault value for port: " + *port)  
}
```

- execute the program with the argument "--help" to see the possible arguments and the suggestions

Summary

- Compiled (cross compiler for OS: Mac, Windows, Linux and CPU: ARM, x86 + Amd64)
- Static linker (Single Binary) -> Ideal for Docker containers
- Focus on fast compile times (the entire Go codebase compiles in <20 seconds)
- Simple (less keywords like C - 25/32) and Orthogonal
- Strong type system with runtime support (Reflection, Dynamic Types)

46

Exercise 2.2

[github.com/s-macke/concepts-of-programming-languages/blob/master/docs/exercises/
Exercise2.2.md](https://github.com/s-macke/concepts-of-programming-languages/blob/master/docs/exercises/Exercise2.2.md) (<https://github.com/s-macke/concepts-of-programming-languages/blob/master/docs/exercises/Exercise2.2.md>)

47

Thank you

Sebastian Macke

Rosenheim Technical University

Sebastian.Macke@th-rosenheim.de (<mailto:Sebastian.Macke@th-rosenheim.de>)

<https://www.qaware.de> (<https://www.qaware.de>)

